BOWIE STATE UNIVERSITY

COSC 369 | Software Engineering I

Faculty Teaching and Office-Hours Scheduler

Project report / working C implementation
Source-based project demonstration: Fall 2026 CTEC workbook; COSC 369 syllabus; COSC 369 project assignment.

The included program and outputs are a reproducible demonstration. Class schedules are extracted from provided sources; faculty office hours are reported only where explicitly provided. Missing information is never presented as confirmed.

1. Problem and requirements

The assignment requires a modular program in C that reads exactly two files: (1) a course/class schedule and (2) faculty information. The program must generate a professor teaching schedule and office-hours schedule, compile reusable code into a static library, and use a Makefile for automated compilation and output generation. A README, generated samples, code, input files, and a PDF explanation are also required.

The COSC 369 syllabus frames the project around software design, data input, analysis, reporting, and documenting a working system. The assignment additionally demonstrates optional Python extraction and Prolog knowledge representation.

2. Input sources and provenance

Runtime file	Source and interpretation
data/courses.csv	56 CTEC spreadsheet rows, plus one distinct COSC 369 row from syllabus / project sample.
data/faculty.csv	28 CTEC instructor names with office hours unknown, plus Dr. Appolo Tankeh with explicitly stated office hours.

The CTEC workbook identifies its instructor column as “Instructor 2025” despite the Fall 2026 filename. Those course/instructor associations are therefore source observations, not verified Fall 2026 faculty assignments. The COSC 369 syllabus has a Spring 2026 heading and Fall 2026 weekly dates; this conflict was not silently corrected.

The two files are stored locally for offline reproducibility. C uses fopen/fgets to read both. The workbook was converted once into CSV to keep the C runtime independent of Excel packages. Python utilities run only after the C program produces output.

3. Static library, modules, and Makefile

A static library is an archive of compiled object files linked into an executable at build time. Here src/scheduler.c is compiled to build/scheduler.o; “ar rcs libschedule.a build/scheduler.o” creates the archive. The main module links with “gcc ... -L. -lschedule -o scheduler”. The Makefile records file dependencies and recompiles changed source files. “make test” builds, executes, and verifies the output; “make report” additionally generates Prolog facts and a readable schedule.

The modular design keeps command-line control in main.c, public declarations in scheduler.h, reusable CSV parsing/validation/report generation in scheduler.c, optional Prolog export in generate_facts.py, and verification in tests/test_outputs.py.

4. Scheduling and conflict rules

The scheduler writes every course record with instructor, days, start/end time, room, source, and a validation status. For each faculty office-hours record with specified times, it finds classes for the same exact instructor name. If an office-hours interval overlaps any class on a shared weekday, the slot is marked CONFLICT; missing office hours remain “unknown - not supplied”. The program does not infer working hours or invent faculty availability.

Meetings without start/end times are labeled asynchronous / no fixed meeting; malformed or non-increasing clock times are marked for manual review, not repaired by guessing. These checks respect the assignment’s statement that instructors should not be interrupted during classes.

5. Sample answers and output

Question	Supported answer
Who teaches COSC 369?	Dr. Appolo Tankeh
When is COSC 369?	Tuesday / Thursday, 9:30 AM–10:45 AM
Where is COSC 369?	CS 108 (project sample, also syllabus office location)
What are COSC 369 office hours?	Tuesday / Thursday, 10:45 AM–1:00 PM

The sample project assignment also illustrates COSC 631 and COSC 330 classes and Monday/Wednesday office hours. These are display examples only, so this implementation does not hard-code them as instructor-confirmed data.

6. Python and Prolog roles

The C executable is the required scheduling application. As optional postprocessing, Python reads C-generated CSV output and creates Prolog clauses. PySwip provides access to a separately installed SWI-Prolog engine. A query such as who_teaches('COSC 369',Professor) retrieves the instructor from generated facts. The C scheduler runs independently of Python, PySwip, or SWI-Prolog.

7. Testing results and limits

A clean GCC/Make build was performed. The project tests verified 57 course rows, one COSC 369 row, the expected COSC 369 start time, one source-confirmed nonconflicting office-hour record, two asynchronous sections, and retention of anomalous source times. The core scheduler completed without a teaching overlap in the provided COSC 369 office-hours slot.

Limitation: the source materials do not supply office hours for CTEC instructors. The generated office-hours file therefore flags those instructors as unknown, rather than promising an automatically assigned timetable. A complete institution-wide office-hours assignment would require approved availability and allocation rules.

8. Deliverables and instructions

The accompanying ZIP contains source files, headers, Makefile, static library, compiled executable (Linux), exactly two C input files, optional Python scripts, Prolog knowledge base, generated schedule CSV files, text summary, README, tests, and this report. On Linux/WSL: run **make test** and **make report**. Optional Prolog queries require SWI-Prolog and PySwip.

9. References: supplied materials

1. COSC 369 – Software Engineering First Project Assignment, provided DOCX, pp. 1–5. 2. COSC 369 – Software Engineering I, Dr. Appolo Tankeh syllabus, provided DOCX, pp. 1–9. 3. CTEC Fall 2026 Schedule – Share, provided XLSX, Sheet1. No external source is used to alter the supplied faculty facts.